

MLOps End-to-End: Machine Learning in Data Science ·

Bao phủ: Bài 15–19 (slides)

Hình thức: Tự làm theo dự án xuyên suốt ·

Dự án: repo bc_mlops/, dataset Breast Cancer (sklearn), chạy CPU · **Thời lượng:** 8–10 giờ

Quy tắc (đọc một lần)

1. Phiếu này **không cung cấp code mẫu**. Bạn tự viết code từ đặc tả.
2. Mỗi bước: **làm** → **báo cáo con số CHÍNH XÁC bạn nhận được** → **giải thích**.
3. Câu  **phản biện** phải trả lời bằng lời của bạn, **gắn với số liệu bạn tự chạy** (không chép slide).
4. **Chống phụ thuộc AI:** bài chấm theo *lập luận + số liệu thật + vấn đáp*, **không chấm code copy được**. Dán thẳng câu hỏi cho AI sẽ khiến câu trả lời không khớp số liệu của bạn và không qua được vấn đáp.

Cách nộp

- Repo Git (bc_mlops/) + file ANSWERS.md (mỗi mục ghi rõ **số liệu của bạn**).
 - 3 ảnh chụp: MLflow **Compare**, Swagger **/docs**, **báo cáo drift**.
 - Sẵn sàng **vấn đáp 5–7 phút** (ngân hàng câu hỏi ở cuối).
-

Module 0 — Khởi tạo dự án & Git (Bài 18, 15)

Bước:

1. Tạo cấu trúc repo chuẩn: data/ src/ app/ tests/ models/ logs/ config/ .github/workflows/.
2. git init; viết .gitignore loại models/*.joblib, dữ liệu lớn, .venv/, secrets.
3. Tạo nhánh main / develop / feature/*; làm việc trên feature/*; commit thường xuyên với message rõ nghĩa.

Nộp: cây thư mục · git log --oneline · nội dung .gitignore.

Phản biện:

- Vì sao versioning trong ML phải gồm **code + data + model** (khác phần mềm truyền thống chỉ code)? Nêu **một hậu quả cụ thể** nếu chỉ version code.
 - Vì sao **không** commit models/*.joblib và secrets? Kể tên công cụ thay thế để version data/model.
-

Module 1 — ML Pipeline & chống rò rỉ dữ liệu (Bài 18)

Bước:

1. Nạp Breast Cancer; `train_test_split` có `stratify`, `seed` cố định.
2. Xây Pipeline = (scaler) + (estimator). Fit → dự đoán → tính **Accuracy, F1, ROC-AUC + CV 5-fold**.
3. 🧠 **Trước khi chạy**: ghi dự đoán ROC-AUC của bạn. Sau đó đối chiếu.
4. 🐛 **Săn lỗi**: cố tình scale toàn bộ X trước khi split, đo lại ROC-AUC. **Ghi chênh lệch bạn quan sát được**.

Nộp: 4 chỉ số (số của bạn) · chênh lệch giữa bản leakage và bản đúng.

🔍 Phản biện:

- Cơ chế nào của Pipeline **chống data leakage** khi cross-validate? Giải thích qua thứ tự `fit/transform`.
- Nếu đổi estimator sang **RandomForest**, `StandardScaler` còn ý nghĩa không? Vì sao (dựa vào cách cây chia nhánh)?
- Vì sao bug ở bước 4 lại cho điểm **cao hơn nhưng là điểm giả**? Điều gì xảy ra khi mô hình gặp dữ liệu thật?

Module 2 — Theo dõi thí nghiệm với MLflow (Bài 19)

Bước:

1. Bọc huấn luyện trong một MLflow run; log **params, metrics, artifact model**. Thử `mlflow.autolog()`.
2. Chạy **≥ 3 cấu hình** (đổi model/hyperparameter). Mở UI (`mlflow ui`), dùng **Compare**.
3. **Đăng ký** model tốt nhất vào Registry; chuyển trạng thái **Staging** → **Production**.

Nộp: ảnh **Compare** (thấy `run_id/timestamp` của bạn) · tên model đã đăng ký · số liệu run tốt nhất.

🔍 Phản biện:

- `roc_auc` (metric) và thư mục `model/` (artifact) được lưu ở **hai kho nào** (Bài 19)? Vì sao tách kho giúp nhóm mở rộng quy mô?
- Load model theo `models:/Name/Production` **hơn gì** so với load theo đường dẫn file? Gắn với **rollback** và **A/B test**.
- MLflow đảm bảo **reproducibility** bằng cách log những gì? **Thiếu thứ nào** thì vẫn không tái lập được kết quả?

Module 3 — Triển khai REST API (Bài 16)

Bước:

1. Serialize **toàn bộ pipeline** bằng joblib, đặt tên có version. Load lại, assert dự đoán khớp.
2. Viết API (FastAPI): POST /predict nhận đặc trưng → trả nhãn + confidence; thêm GET /health, GET /model-info.
3. Chạy uvicorn; test bằng curl **≥ 3 mẫu thật**; mở /docs.
4. 🐛 **Săn lỗi**: lưu **chỉ estimator** (bỏ scaler) rồi phục vụ dữ liệu thô → **mô tả sai lệch bạn thấy**; gọi tên hiện tượng.

Nộp: log curl 3 mẫu (kèm nhãn thật) · ảnh /docs.

🔍 Phản biện:

- **Training-serving skew** là gì, vì sao lỗi này **âm thầm** (không có exception)? **Nêu quy tắc một câu** để phòng tránh.
- 4 kiểu triển khai (real-time / batch / streaming / edge): chọn kiểu cho **(a)** chấm điểm ảnh y khoa hàng loạt ban đêm, **(b)** chặn gian lận thẻ theo thời gian thực. Giải thích.
- Vì sao mô hình ML thường dùng **POST** chứ không **GET** để nhận đặc trưng đầu vào?

Module 4 — Giám sát mô hình & phát hiện Drift (Bài 17)

Bước:

1. 📝 **Tính tay PSI** trên một bảng 4-bin (tự tạo theo mẫu Expected/Actual). Ghi rõ từng cột.
2. **Tự viết hàm PSI**; kiểm chứng kết quả code **khớp** với tính tay.
3. Mô phỏng 2 kịch bản production: **không-drift** & **có-drift** (dịch phân phối 1 đặc trưng). Tính **PSI + KS-test** cho cả hai; áp ngưỡng **PSI ≥ 0.2**.
4. Bổ sung **ghi log dự đoán** ở API: timestamp, features, prediction, confidence, model_version.
5. 🌐 **Trước khi chạy bước 3**: dự đoán PSI/KS ở mỗi kịch bản.

Nộp: bảng tính tay (có PSI) · output 2 kịch bản (số của bạn) · 3 dòng log mẫu.

🔍 Phản biện:

- Phân biệt **data drift** vs **concept drift** bằng ví dụ TMĐT Việt Nam **của bạn**. Loại nào phát hiện được **mà không cần nhãn thật**?
- Vì sao mô hình “**hồng trong im lặng**”? Vì sao **theo dõi phân phối đầu ra** vẫn hữu ích khi nhãn về rất trễ?

- Đặt ngưỡng cảnh báo “**theo tác động kinh doanh, không chỉ thống kê**”: cho **hai ngưỡng khác nhau** cho cùng một mức PSI ở hai bài toán khác nhau.
- Theo quy trình giám sát 6 bước (Bài 17): khi alert kêu thì làm gì tiếp? Khi nào **retrain theo lịch** vs **retrain theo trigger** vs **rollback**?

Module 5 — Kiểm thử & CI/CD (Bài 18, 15)

Bước:

1. Viết **≥ 4 unit test** (pytest): shape đầu ra, nhãn nhị phân, proba $\in [0,1]$, **edge case NaN**. Chạy kèm coverage.
2. Viết **GitHub Actions**: chạy test + train thử trên mẫu nhỏ mỗi lần push.
3. 🐛 **Sẵn lỗi**: cho một test “luôn pass nên vô dụng” (chấm trên tập đã fit, ngưỡng 0.5). Nêu **2 vấn đề** và viết lại.

Nộp: file test · ci.yml · kết quả pytest (số test pass, coverage).

🔍 Phản biện:

- CI bắt được **loại lỗi nào**? Loại “hỏng” nào CI **không** bắt được mà chỉ Module 4 (giám sát) mới thấy? Nối với vòng đời MLOps (Bài 15).
- CI/CD (phần mềm) khác **CT — Continuous Training** ở đâu? Điều gì **kích hoạt** CT?
- Vì sao unit test nên **tách train/test ngay trong test**, không chấm điểm trên dữ liệu đã fit?

Module 6 — Tổng hợp & Độ trưởng thành MLOps (Bài 15)

Bước:

1. Điền bảng: mỗi **giai đoạn vòng đời MLOps** ↔ **thành phần bạn đã làm** (Module 0–5).
2. Tự xếp hệ thống ở **Level 0 / 1 / 2**; nêu **bằng chứng**.
3. Nêu **một bước cụ thể** để lên level kế tiếp + **lý do ưu tiên**.

Nộp: bảng ghép nối · đoạn tự đánh giá 5–7 câu.

🔍 Phản biện:

- Cho **một ví dụ từ chính bài làm** cho mỗi khác biệt DevOps vs MLOps: **Testing** và **Maintenance**.
- Trong toàn hệ thống, **mắt xích yếu nhất về reproducibility** là gì? Vì sao?

Ngân hàng câu hỏi vấn đáp (trả lời không mở tài liệu)

1. Vì sao “xây được mô hình ≠ giải quyết bài toán”? MLOps lấp khoảng trống nào?
2. Ba thành phần được version trong MLOps là gì, và vì sao thiếu một trong ba thì mất reproducibility?
3. Đặt StandardScaler trong Pipeline chống leakage bằng cơ chế nào?
4. Vì sao phải lưu **cả pipeline** thay vì chỉ estimator?
5. POST vs GET cho ML inference — chọn cái nào, vì sao?
6. $PSI \geq ?$ thì báo drift; PSI phát hiện loại drift nào **không cần** nhãn?
7. “Mô hình hỏng trong im lặng” nghĩa là gì và hệ quả vận hành?
8. CT khác CI/CD ở điểm cốt lõi nào?
9. Registry giúp rollback và A/B test bằng cách nào?
10. Hệ thống của bạn đang ở maturity level mấy, thiếu gì để lên tiếp?

Ma trận phủ kiến thức

Cụm kiến thức (theo bài)	Module phủ
Bài 15 — Why/What MLOps, DevOps vs MLOps	M6, vấn đáp Q1
Bài 15 — Vòng đời & thành phần (VC, CI/CD/ CT , Registry, Monitoring, Reproducibility)	M0, M2, M5, M6
Bài 15 — Maturity Level 0/1/2, hệ sinh thái công cụ	M6
Bài 16 — Deployment types, workflow 6 bước	M3
Bài 16 — Serialization; training-serving skew	M3 (bước 1, 4)
Bài 16 — REST API, Flask/FastAPI, Pydantic, POST/GET, curl	M3
Bài 17 — Data drift (PSI/KS, ngưỡng 0.2) & Concept drift	M4
Bài 17 — Performance decay, prediction-distribution monitoring	M4 (phản biện)
Bài 17 — Logging predictions; quy trình 6 bước; retrain/rollback	M4
Bài 18 — Pipeline (transformer/estimator), chống leakage, ưu/nhược	M1
Bài 18 — Git workflow, repo structure, versioning ML	M0
Bài 18 — Unit testing (pytest, edge case, coverage), CI/CD	M5
Bài 19 — MLflow 4 components; tracking API; backend/artifact store	M2
Bài 19 — Model Registry (Staging/Production/Archived), reproducibility	M2